

Java Fundamentals

Placement Training Program

Syntax & Data Types

Java is a statically-typed, compiled language — every variable's type is fixed at declaration and checked before the program ever runs, which catches many bugs at compile time that a dynamically-typed language would only catch (or silently mishandle) at runtime. Java's primitive types (int, double, boolean, char, etc.) hold raw values directly and are not objects, while reference types (String, arrays, and any class) hold a reference to an object on the heap. Understanding this distinction explains a lot of Java's behavior, including why comparing objects with `==` checks reference identity, not value equality (use `.equals()` for that).

Control Flow

Control flow statements — if/else, switch, for, while, do-while — determine the order code executes in. A for loop is the natural choice when the number of iterations is known upfront (like iterating a fixed-size array); a while loop suits situations where the loop should continue based on a condition that isn't a simple counter (like reading input until a sentinel value appears). Java's enhanced for-each loop (`for (Type item : collection)`) is preferred whenever you don't need the index itself, since it's shorter and eliminates an entire class of off-by-one indexing bugs.

Object-Oriented Programming Basics

Java is built around four core OOP principles, each solving a real design problem:

- Encapsulation — bundling data and the methods that operate on it inside a class, and hiding internal state behind private fields with public getter/setter methods, so external code can't put an object into an invalid state directly.
- Inheritance — a class can extend another, inheriting its fields and methods, used to model an 'is-a' relationship (a `SavingsAccount` is a kind of `Account`).
- Polymorphism — code written against a parent type can work with any subtype without modification, since a subtype can be used anywhere its parent type is expected.
- Abstraction — exposing only what's necessary through interfaces or abstract classes, hiding implementation detail the caller doesn't need to know about.

Arrays & Strings

Arrays hold a fixed number of elements of the same type, indexed from 0; once created, an array's length can never change (Java's `ArrayList`, covered in the next module, is the resizable alternative built on top of arrays). Strings in Java are immutable — every operation that appears to modify a `String` (like concatenation) actually creates a new `String` object, leaving the original unchanged. This immutability is a frequent source of confusion for beginners (why doesn't `str.toUpperCase()` change `str` itself?) and also the reason repeated string concatenation in a loop is inefficient — `StringBuilder` exists specifically to build up a string efficiently without creating a new object on every append.

Exception Handling

Exceptions represent errors that occur during program execution — Java forces you to acknowledge and handle certain error conditions (checked exceptions, like `IOException`) at compile time, while others (unchecked/runtime exceptions, like `NullPointerException`) are not required to be explicitly caught but still occur if the code has a bug. The `try/catch/finally` structure lets you attempt an operation, handle specific failure types differently, and guarantee cleanup code runs (`finally`) whether or not an exception occurred. Catching an exception only to silently ignore it (an empty catch block) is one of the most common and dangerous anti-patterns in Java code — it hides real bugs from ever surfacing.

Learn More

Oracle — The Java Tutorials

<https://docs.oracle.com/javase/tutorial/>

GeeksforGeeks — Java Programming Language

<https://www.geeksforgeeks.org/java/>

freeCodeCamp YouTube Channel

<https://www.youtube.com/@freecodecamp>